UNIVERSIDAD RAFAEL LANDÍVAR

Facultad de Ingeniería — Campus Central

Departamento de Ingeniería en Informática y Sistemas

Curso: Programación Web | Catedrático: Ing. Ronald Leonel Ayala Garrido

DOCUMENTACIÓN TÉCNICA DE INGENIERÍA Y MANUAL DE USUARIO

ReserveHub — Sistema Web Integral de Reservas de Espacios Universitarios

Guatemala, noviembre de 2026

---

1. IDENTIFICACIÓN DEL EQUIPO Y PROYECTO

Carné	Nombre del Integrante	Rol en el Proyecto
1109722	Walter José Herrera Salvatierra	Desarrollo Frontend, Componentes UI y Ruteo SPA
1218122	Boris Garzona	Diseño de Base de Datos y Modelado Relacional Prisma
1045222	Jonathan García	Arquitectura de API REST, Controladores y Despliegues
1228722	Samuel Ruiz	Lógica de Negocio RN-01 y Detección de Colisiones
1230222	Giancarlo Ortega	Seguridad, Hashing, Autenticación JWT y Middlewares
2477123	Mario López	Infraestructura Docker, Contenerización y Testing Vitest

2. ENLACES OFICIALES DEL DESPLIEGUE EN PRODUCCIÓN

La plataforma se encuentra completamente funcional, desacoplada y disponible en la nube mediante los siguientes enlaces públicos:

- **Aplicación Web Frontend (Vercel):** https://proyecto-final-progra-web-theta.vercel.app/
- **Módulo de Reservas de Usuario:** https://proyecto-final-progra-web-theta.vercel.app/my-reservations

- **API REST Backend en Vivo (Render):** https://reservehub-backend-881f.onrender.com/api/health
- **Repositorio Oficial de Código (GitHub):** https://github.com/JonyGarciaaa/ProyectoFinalPrograWeb

Credencial Admin:	admin@reservehub.com | *Contraseña:* Admin123!
Credencial Usuario:	jonathan@reservehub.com | *Contraseña:* User123!

3. EXPLICACIÓN DE LA ARQUITECTURA DEL SISTEMA

El sistema ReserveHub fue concebido bajo un patrón arquitectónico de desacoplamiento total en tres capas independientes (Three-Tier Architecture), asegurando alta disponibilidad, mantenibilidad y escalabilidad horizontal:

3.1 Capa de Presentación (Frontend SPA)

Construida con **React 19**, **TypeScript** y empaquetada mediante **Vite**. Implementa enrutamiento del lado del cliente con **React Router v7** y diseño responsivo basado en **Tailwind CSS**. Toda la comunicación con el servidor se realiza de manera asíncrona mediante un cliente centralizado de API (Axios/Fetch) con interceptores que inyectan credenciales de sesión en cada solicitud.

3.2 Capa de Lógica de Negocio y Servicios (Backend REST API)

Desarrollada sobre **Node.js 22 LTS** utilizando el framework **Express 5** y tipado estático estricto con **TypeScript**. La arquitectura del servidor sigue una separación clara de responsabilidades:

- **Capa de Rutas (Routes):** Mapeo semántico de endpoints bajo estándares RESTful (/api/auth, /api/spaces, /api/reservations).
- **Capa de Middlewares:** Verificación de tokens JWT (authMiddleware), control de acceso basado en roles (roleMiddleware: ADMIN vs USER), validación tipada de esquemas con **Zod** (validateMiddleware) y manejador global de errores tipados.
- **Capa de Controladores (Controllers):** Orquestación de peticiones, validación de reglas de negocio y formateo estandarizado de respuestas JSON (código HTTP, mensaje y payload).
- **Capa de Persistencia (ORM):** Gestión transaccional mediante **Prisma Client v6**, garantizando operaciones ACID e inmunidad contra inyección SQL.

3.3 Capa de Datos (PostgreSQL 16 en la Nube)

Alojada en una instancia administrada de **PostgreSQL 16** en Render. El esquema define claves foráneas con integridad referencial, índices compuestos sobre columnas de búsqueda crítica (spaceId, startTime, endTime) y restricciones de unicidad compuestas para garantizar la coherencia de datos.

3.4 Regla de Negocio Crítica: Prevención Matemática de Solapamiento (RN-01)

Para evitar que un espacio sea reservado dos veces en horarios superpuestos, el sistema implementa la fórmula matemática de intersección de intervalos semiabiertos dentro de una transacción de base de datos:

Existe colisión si y solo si: (Inicio_Existente < Fin_Solicitado) Y (Fin_Existente > Inicio_Solicitado)

Cualquier reserva en estado CANCELLED o REJECTED es excluida automáticamente del chequeo, permitiendo reutilizar inmediatamente el espacio liberado.

4. MANUAL DE USUARIO DEL SISTEMA RESERVEHUB

A continuación se detallan las interfaces del sistema, explicando el propósito de cada componente, las acciones disponibles para el usuario y capturas de pantalla de la aplicación en producción.

4.1 Pantalla de Inicio (Landing Page)

Es el punto de entrada a la plataforma. Presenta la propuesta de valor del sistema, estadísticas en tiempo real y accesos directos hacia el catálogo de espacios y los formularios de autenticación.



Figura 1. Pantalla de inicio de ReserveHub desplegada en producción en Vercel.

4.2 Catálogo Interactivo de Espacios

Permite a los miembros de la comunidad universitaria visualizar la oferta de instalaciones (aulas magnas, laboratorios de computación, salas de conferencias y cubículos de estudio). La vista incluye un buscador por palabras clave, filtro dinámico por capacidad mínima de personas y selector de fecha para verificar disponibilidad inmediata.

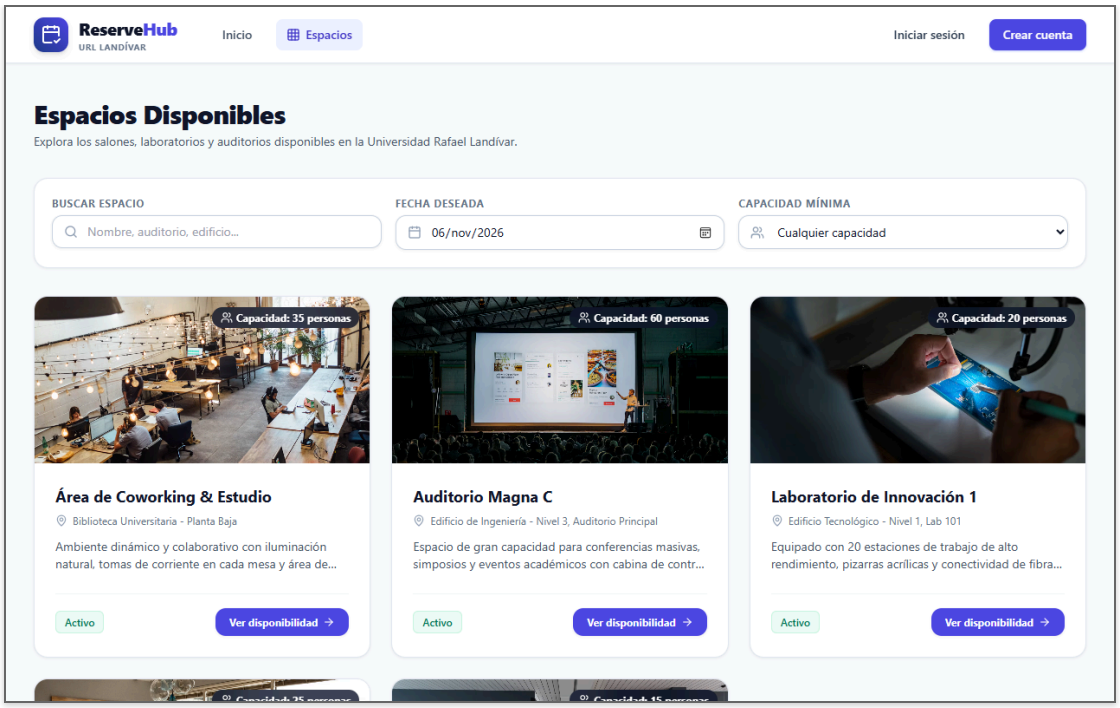


Figura 2. Catálogo de espacios universitarios con tarjetas informativas y badges de estado.

4.3 Registro de Nuevos Usuarios

Formulario para que nuevos estudiantes y docentes puedan crear su cuenta institucional. Los datos son validados en el cliente y en el servidor con Zod (correo con formato válido, contraseña de al menos 6 caracteres con números y letras). Las contraseñas son hasheadas con bcrypt (factor de costo 10) antes de persistir en PostgreSQL.

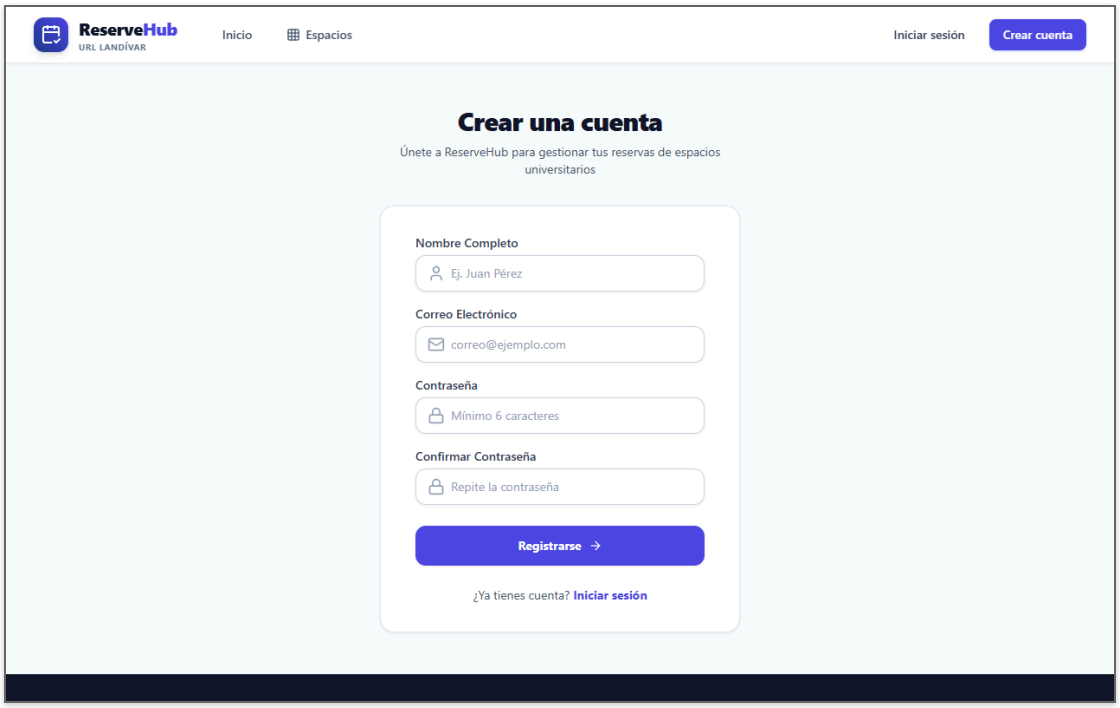


Figura 3. Formulario de registro con validaciones de seguridad en tiempo real.

4.4 Inicio de Sesión y Autenticación Segura

Permite a los usuarios identificarse en el sistema. Al autenticarse correctamente, el servidor genera un JSON Web Token (JWT) firmado criptográficamente, estableciendo la sesión y habilitando la navegación hacia las rutas protegidas.

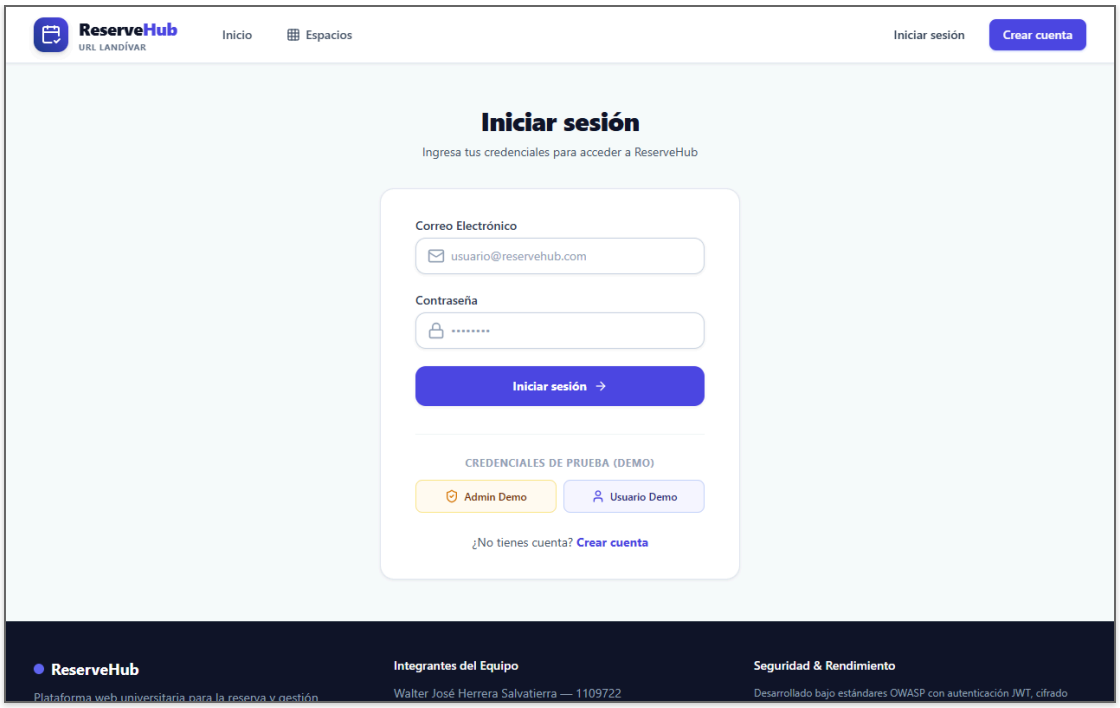


Figura 4. Pantalla de autenticación con opción de credenciales precargadas para pruebas.

4.5 Módulo de "Mis Reservas" (/my-reservations)

Ubicado en la ruta directa /my-reservations, este módulo es el centro de control del usuario autenticado. Ofrece las siguientes funcionalidades:

- **Listado de Reservas:** Muestra el historial completo de solicitudes realizadas por el usuario con nombre del espacio, fecha, horario y motivo.
- **Etiquetas de Estado en Tiempo Real:** Cada reserva cuenta con un indicador visual: *CONFIRMED* (Confirmada), *PENDING* (Pendiente de aprobación), *CANCELLED* (Cancelada) o *REJECTED* (Rechazada).
- **Filtros por Pestañas:** Permite alternar entre "Todas", "Próximas", "Pasadas" y "Canceladas".
- **Cancelación Segura:** Botón que permite al titular cancelar una reserva futura, liberando instantáneamente el espacio en la base de datos para otros usuarios.

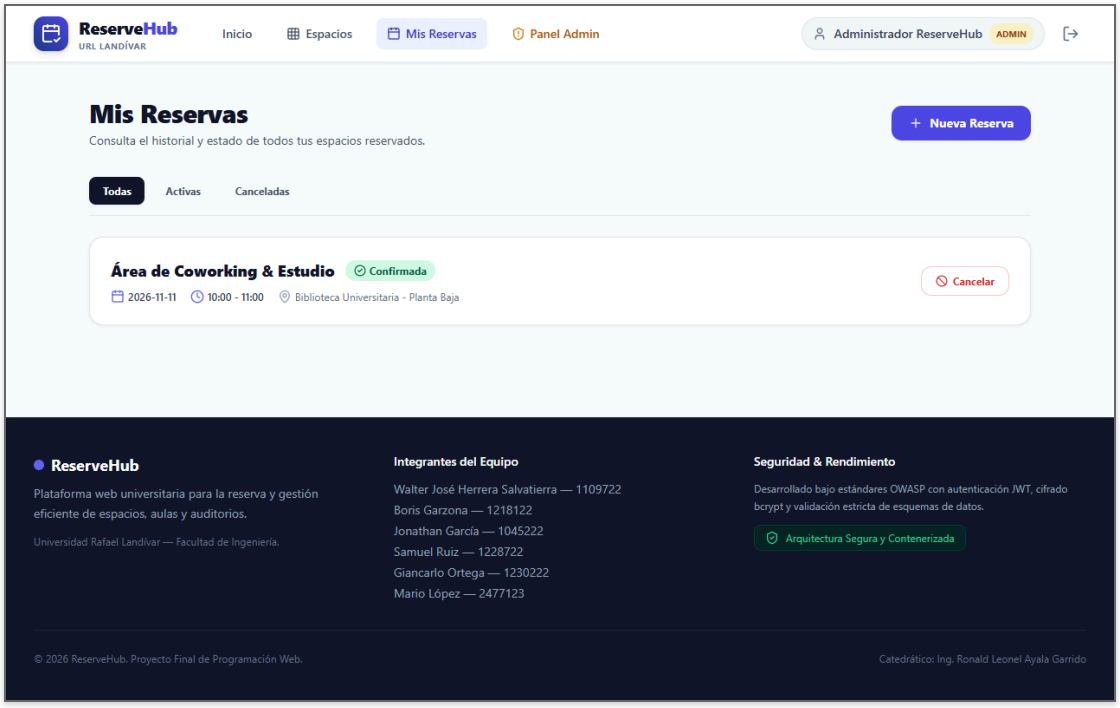


Figura 5. Módulo de 'Mis Reservas' (/my-reservations) con filtros de estado y gestión de cancelaciones.

4.6 Formulario de Reserva y Validación de Horarios

Al seleccionar un espacio, el usuario ingresa al formulario de reservación donde elige la fecha en un calendario interactivo. El sistema consulta en la base de datos los horarios ocupados y muestra dinámicamente las franjas horarias libres. Al enviar el formulario, el backend ejecuta la validación RN-01 de no solapamiento antes de confirmar el registro.

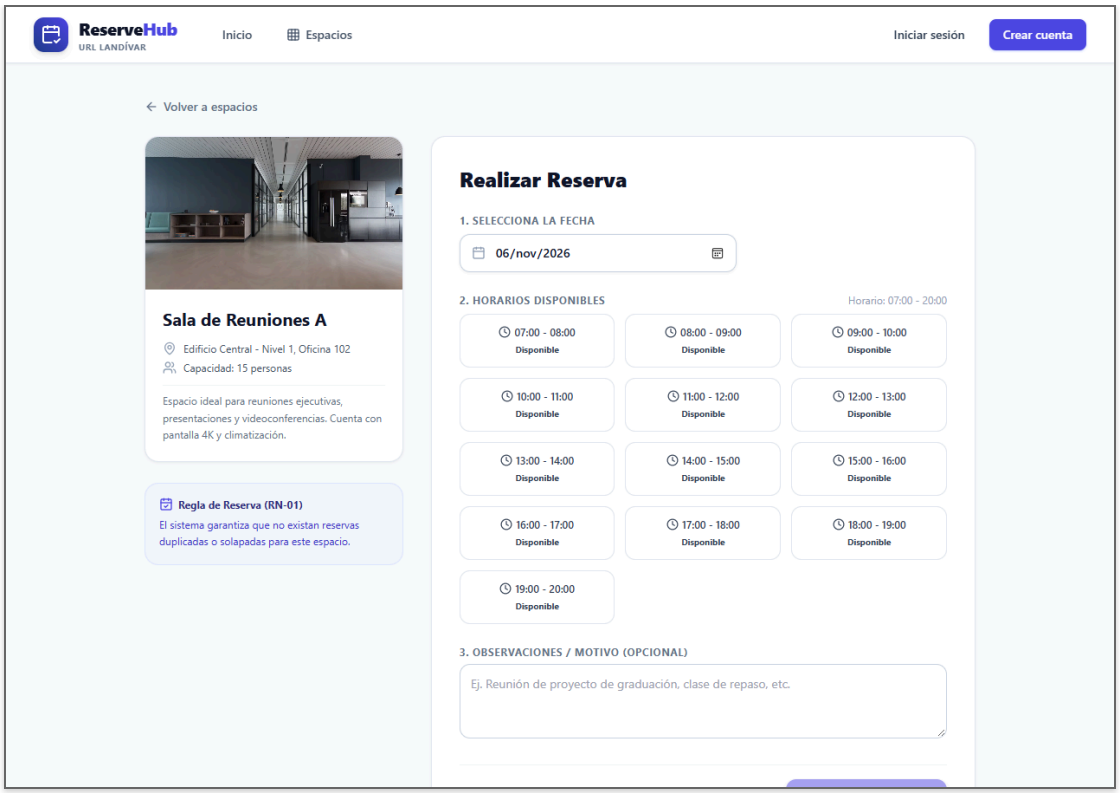


Figura 6. Interfaz de reserva interactiva con selector de fecha y bloques horarios.

4.7 Panel de Control Administrativo (/admin)

Interfaz accesible únicamente para usuarios con rol **ADMIN**. El sistema protege esta ruta a nivel de navegación en React y mediante middleware de autorización en el API. Desde este panel, el administrador puede:

- **Gestionar Espacios:** Crear nuevas instalaciones, editar nombres, capacidad, ubicación, o pausar temporalmente un aula por mantenimiento.
- **Supervisar Reservas Institucionales:** Visualizar las solicitudes de todos los usuarios de la universidad, con potestad para aprobarlas, rechazarlas o cancelarlas.
- **Monitoreo de Métricas:** Estadísticas de ocupación general de las instalaciones de la universidad.

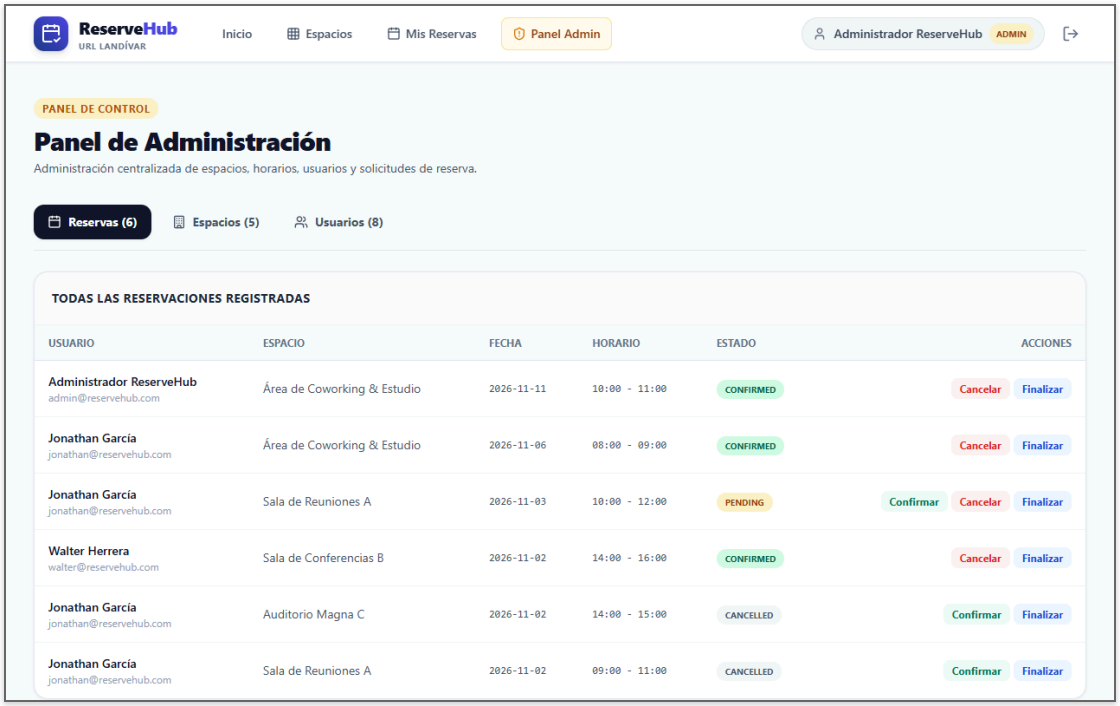


Figura 7. Panel de control administrativo exclusivo para gestión de instalaciones y auditoría de reservas.

5. GUÍA DE EJECUCIÓN LOCAL CON DOCKER (REPRODUCIBILIDAD)

El sistema cuenta con configuración completa de Docker y Docker Compose para su levantamiento en local sin necesidad de instalar dependencias de Node.js o PostgreSQL en la máquina del evaluador:

1. Clonar el repositorio: `git clone https://github.com/JonyGarciaaa/ProyectoFinalPrograWeb.git`
2. Acceder a la carpeta: `cd ProyectoFinalPrograWeb`

3. Ejecutar la orquestación: `docker compose up --build`
4. Abrir el navegador en `http://localhost:5173` para el frontend y `http://localhost:3000/api/health` para el backend.

6. CONCLUSIONES TÉCNICAS

El proyecto ReserveHub cumple de manera integral con los lineamientos de la materia Programación Web, demostrando la integración efectiva de una arquitectura moderna basada en componentes, tipado estricto con TypeScript, persistencia relacional transaccional y buenas prácticas de seguridad e infraestructura en la nube.